FULL-STACK DEVELOPER · BACKEND & INFRA

# 경력 포트폴리오


이 름 윤경록

연락처 010-2967-2625




## 자기소개 백엔드·인프라에 강한 풀스택 개발자

센서 데이터 수집부터 관제 화면, 클라우드 인프라까지 직접 만들어 온 풀스택 개발자입니다. Java·Python·NestJS 백엔드, React 화면, AWS·Docker 인프라부터 YOLO·Kubeflow·Gemini까지 필요한 기술을 익혀 실무에 적용해 왔습니다. 문제의 원인을 끝까지 파고들고, 누구나 이어받을 수 있게 기록하고 자동화합니다.

| | | |
|---|---|---|
| 기본 정보 | 이 메 일 | tjsal890000@naver.com |
| | GitHub | github.com/ygrhash |
| | 데 모 | aiops.ygrhash.dev (개인 프로젝트 라이브 데모) |
| | 주요 기술 | Java · Spring Boot · Python · NestJS · PostgreSQL · AWS · Docker · Nginx · Prometheus · Grafana |

| | | |
|---|---|---|
| 경 력 | 2025.06 – 2026.08 (1년 3개월) | **그랜드파워솔루션** · 주임연구원 · 정규직<br>SI / 풀스택 개발 / 서버관리<br>• 한화오션 절단공장 야드 관제 시스템 개발 (2025.06 – 2026.08)<br>• 한화오션 전처리 공장 QR 인식 시스템 개발 (2025.06 – 2026.08)<br>• K-Cloud GPU 할당 Cloud IDE 구현 (2026.07 – 2026.08) |
| | 2022.01 – 2025.05 (3년 5개월) | **(주)제이씨에프테크놀러지** · 주임연구원 · 정규직<br>풀스택 개발 / 서버관리<br>• 맥케어 생체신호 모니터링 플랫폼 개발 (2022.01 – 2025.05)<br>• 관제 대시보드 개발 (2022.01 – 2025.05)<br>• AWS 클라우드 서버 구축 및 운영 (2022.01 – 2025.05)<br>• 온프레미스 서버 및 네트워크 보안 환경 구축 (2022.01 – 2025.05)<br>• 서버 관리 (2022.01 – 2025.05) |
| | 2020.11 – 2021.02 (4개월) | **한국지능기계** · 학생 체험형 인턴<br>• 비전 머신(카메라) 차량 부품 불량 유무 체크 (2020.11 – 2021.02) |

| | | |
|---|---|---|
| 핵심 성과 | 5개국+ | 나라별 AWS 수집 서버 설계·운영 (한국·영국·UAE·일본·독일 등) |
| | 5만 대 | 가상 센서 5만 대 · 48시간 부하테스트로 누락률 1% 미만 검증 |
| | −30% | Saving Plan 도입 제안으로 연간 EC2 비용 절감 |
| | −82% | Spot 인스턴스로 부하테스트 비용 절감 |

## 강점 넓게 익히고, 전체를 책임지고, 끝까지 파고듭니다

| | | |
|---|---|---|
| 강 점 | 넓이 | 필요한 기술이라면 영역을 가리지 않고 익혀 실무에 씁니다. Python 소켓 서버부터 Java·Spring, NestJS, React, AWS·온프레미스, OpenCV·YOLO까지 적용했고, 지금은 Kafka·Kubernetes를 개인 프로젝트로 구축하고 있습니다. |
| | 전체 | 시스템의 처음부터 운영까지 전체 흐름을 책임집니다. 센서 수신, 저장, API, 관제 화면, 모니터링·백업까지 전 구간을 구축했고, 합류한 팀장·팀원에게 시스템의 연혁과 운영 방법을 전했습니다. |
| | 깊이 | 진짜 원인을 찾을 때까지 파고들어 끝까지 해결합니다. DB 커넥션을 늘려도 수집 누락이 계속되자 TCP 연결 유지가 원인임을 찾아 고치고, 가상 센서 5만 대로 48시간 검증했습니다. |

## AI 활용 AI를 서비스와 개발에 직접 씁니다

| | | |
|---|---|---|
| AI 활용 | 실무 · 모델 학습 | 공장 실데이터로 YOLO 기반 QR 영역 탐지 모델 학습 (한화오션 QR 인식) |
| | 실무 · AI 인프라 | Kubeflow 기반 GPU 할당 Cloud IDE 구축 참여, 손글씨 모델 학습으로 검증 (K-Cloud) |
| | 개인 · LLM 연동 | Gemini 운영 리포트 + MCP 서버를 도구로 호출하는 에이전트 (결제 AIOps, 실무와 별개) |
| | 개인 · 개발 생산성 | Claude로 설계 검토 · 코드 리뷰 · 문서화 (결제 AIOps · 포트폴리오) |

## 운영 원칙 장애를 먼저 알고, 미리 검증하고, 빨리 복구합니다

| | | |
|---|---|---|
| 원 칙 | 감지 | 먼저 알아챕니다. Prometheus·Grafana·Zabbix로 상태를 보고, 이상은 Slack으로 즉시 알립니다. |
| | 검증 | 미리 검증합니다. 구조를 바꾸면 가상 센서 5만 대로 48시간 부하테스트 후 반영합니다. |
| | 복구 | 장애가 나도 되살립니다. DB를 SFTP로 백업하고, 온프레미스 저장소는 RAID 1로 이중화합니다. |
| | 자동화 | 손을 덜 탑니다. 인증서 갱신, 백업, 수집 서버 증설을 자동화합니다. |

## 기술 스택 백엔드부터 프론트 · 인프라 · AI까지 직접 다룹니다

| | | |
|---|---|---|
| 기 술 | Backend | 실무 Java, Spring Boot, Spring Batch, Python, Node.js, NestJS |
| | Frontend | 실무 React, Next.js, Redux, TypeScript, Nexacro |
| | Infra · Cloud | 실무 AWS, Docker, Docker Compose, Nginx, Jenkins, Linux<br>개인 Kubernetes(k3s), Terraform |
| | Data · Messaging | 실무 PostgreSQL, MySQL, Oracle<br>개인 Redis, Kafka |
| | Monitoring | 실무 Prometheus, Grafana, Zabbix<br>개인 Tempo |
| | Vision · AI | 실무 OpenCV, YOLO, Kubeflow<br>개인 Gemini API, MCP |

**실무**: 회사 프로젝트에서 사용 · **개인**: 개인 프로젝트(결제 AIOps)에서만 사용

## 학력

| | | |
|---|---|---|
| 학 력 | 2014.02 – 2022.03 | **동명대학교 컴퓨터공학과** · 학사 졸업 · 학점 3.83 / 4.5<br>졸업 작품: 사용자 맞춤형 홈케어 시스템 설계 및 구현 (Latte Panda, Google Dialogflow STT·TTS, Firebase, Arduino 센서) |

## 제가 이 일을 하는 이유

| | | |
|---|---|---|
| 이 유 | 쓰이는 결과 | 제가 만든 시스템이 현장에서 실제로 쓰이고, 사용자의 문제를 해결할 때 가장 큰 보람을 느낍니다. |
| | 필요한 기술은 직접 | 그래서 백엔드·화면·인프라·AI까지 영역을 가리지 않고 필요한 기술을 직접 익혀 문제를 풉니다. |
| | 이어받는 개발 | 누구나 금방 이어받을 수 있게 기록하고 자동화합니다. |

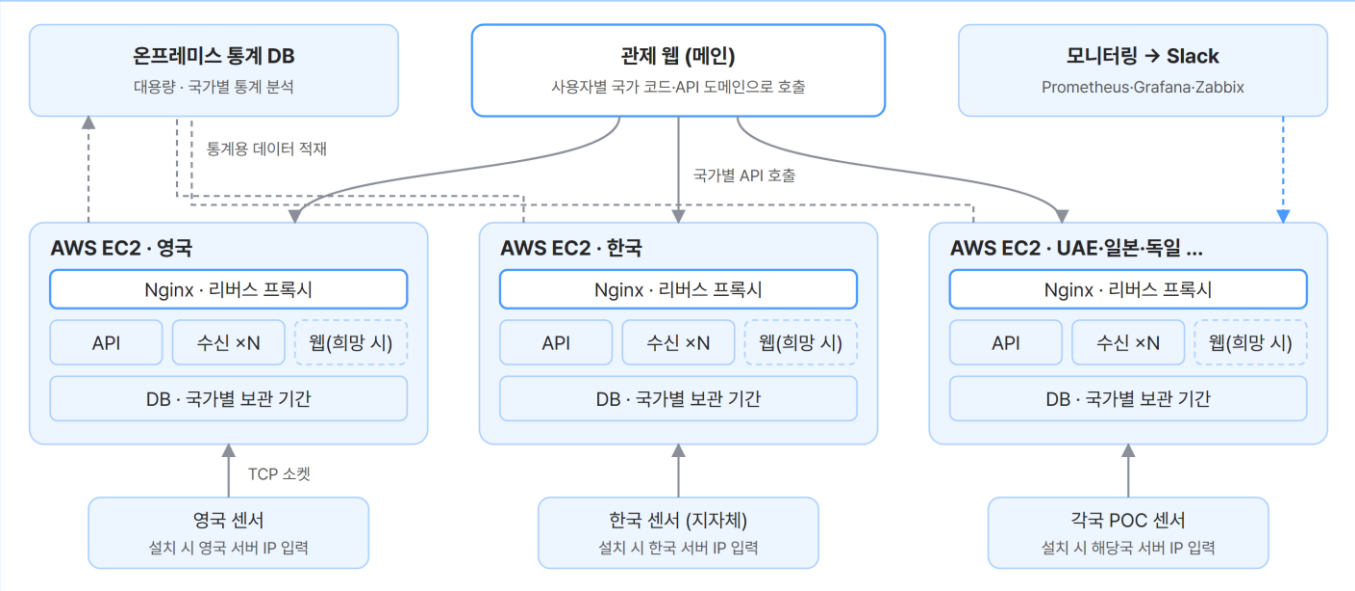
| | |
|---|---|
| 앞으로 하고 싶은 일 | 제가 만든 시스템을 사용자가 불편 없이 쓰고, 안정적으로 계속 운영되는 것이 가장 큰 목표입니다. 한 번 만들고 끝내지 않고 계속 고도화하고, 문서화와 자동화로 새로 합류한 사람도 금방 적응하고 운영하는 사람도 편하게 다룰 수 있는 시스템을 만들고 싶습니다. |

## 1. 맥케어 생체신호 모니터링 플랫폼 (주)제이씨에프테크놀러지 · 주임연구원

| | | |
|---|---|---|
| 프로젝트 개요 | 기 간 | 2022.01 – 2025.05 |
| | 역 할 | 초기 구축 주도 · 인프라·백엔드·관제 웹 프론트 (2년 차부터 합류한 팀장·팀원과 함께 확장) |
| | 규 모 | 실서비스 센서 2,000대 이상 · 한국·영국·UAE·일본·독일 등 5개국+ |
| | 상 태 | 지자체 실서비스 · 해외 POC · 현재 운영 중 |
| | 기 술 | Python · Java · Spring Boot · Spring Batch · NestJS · PostgreSQL · Nginx · Docker · AWS · Prometheus · Grafana · Zabbix · React · Next.js |

| | |
|---|---|
| 프로젝트 설명 | 레이더 센서가 독거 어르신 등 취약계층의 심박·호흡·낙상·재실 여부를 비접촉으로 감지하는 서비스입니다. 입사 첫해 초기 시스템을 직접 구축했고, 2년 차부터 합류한 팀장·팀원과 함께 해외 POC로 확장했습니다. 가용성과 데이터 무결성을 가장 중요하게 다뤘습니다. |

## 운영 구조




나라마다 AWS EC2 인스턴스를 두고 Nginx 뒤에 API·수신 서버·DB를 컨테이너로 운영했습니다. 실제 주소와 서버 이름은 빼고 새로 그린 그림입니다.

| | | |
|---|---|---|
| 나라별로 나눈 이유 | 보관 기간 관리 | POC 단계라 나라마다 데이터를 소규모로 쓰고, 보관 기간을 따로 조절해야 했습니다. |
| | 지연 최소화 | 센서와 같은 나라의 서버가 데이터를 받아, 실시간 감지에 필요한 응답 지연을 줄였습니다. |
| | 나라별 통계 | 통계가 필요한 데이터만 온프레미스 대용량 DB로 모아 국가별로 비교·분석했습니다. |

## 1. 맥케어 · 담당 업무

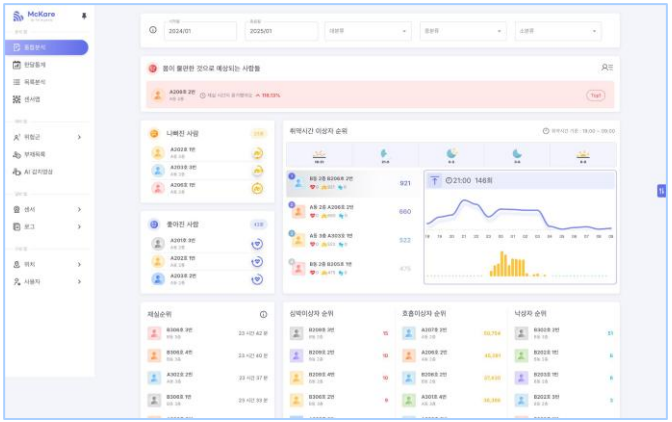
| | | |
|---|---|---|
| 담당 업무 | 데이터 수집 | • 나라별 AWS EC2에 Nginx 리버스 프록시와 API·수신 서버·DB 컨테이너 구성<br>• Python 멀티스레드 소켓 서버로 여러 센서의 동시 연결과 데이터 수신 처리<br>• 수집 컨테이너 수를 가변적으로 늘리고 Nginx 로드밸런서로 분산<br>• 사용자 정보에 국가 코드·API 도메인을 저장해 관제 웹이 해당 나라 API 호출 |
| | 백엔드 | • Spring Boot 백엔드, Spring Batch로 API 호출과 데이터 백업<br>• Java로 센서 데이터 수신과 FFT 알고리즘 구현<br>• NestJS·PostgreSQL 관제 API, API 호출 로깅 공용 모듈 |
| | 인프라 · 비용 | • Saving Plan으로 연간 EC2 비용 약 30% 절감, 부하테스트는 Spot으로 82% 절감<br>• Docker Compose 국가별 일괄 배포, Shell Script로 백업·로그 수집 자동화<br>• 온프레미스 IP·Port 접근 제어, Outbound 제한, RAID 1(12TB 미러링) |
| | 모니터링 | • Prometheus·Grafana·Zabbix로 서버 트래픽·가용량 대시보드 구축<br>• 이상 발생 시 Slack 즉시 알림, Certbot 인증서 자동 갱신, 주·월 단위 백업 |

### 관제 웹 React · Next.js · Redux · NestJS · PostgreSQL

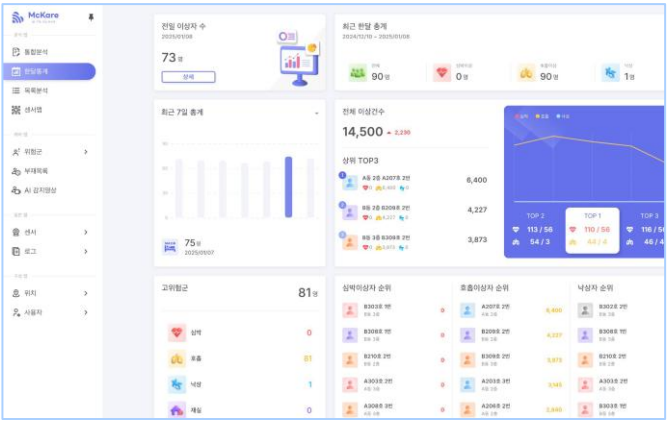
기관 담당자가 센서 상태와 이상 징후를 실시간으로 보고, 기간별 추세를 분석하고, 기관·위치별로 권한을 나눠 관리하는 화면을 직접 개발했습니다.

| | | |
|---|---|---|
| 직접 개발한 기능 | 실시간 관제 | 15초 주기 상태 갱신, 이상 발생 센서 상단 정렬, 당일 이상·낙상 건수와 전일 대비 증감, 이상자별 재실 타임라인과 엑셀 다운로드 |
| | 데이터 분석 | 최근 1개월과 조회 기간을 비교해 재실 시간·이상 건수가 급증·급감한 센서 탐지, 취약시간(18–09시) 이상자 순위, 일·주·월 단위 한달통계 |
| | 위험군 관리 | 사용자가 정하는 저·중·고위험 기준값, 기간별 위험군 목록, 재실 경고 9일 이력 |
| | 권한 · 조직 | 고객키로 소속 기관을 연결하는 회원가입, 4단계 위치 계층, 기관·위치 범위별 모니터링 권한과 일괄 변경 |
| | 공통 UI | 9개 언어 다국어, 라이트·다크 테마, 권한별 메뉴 바로가기, 오늘의 이상 감지 알림, 푸시·소리 알림 설정 |

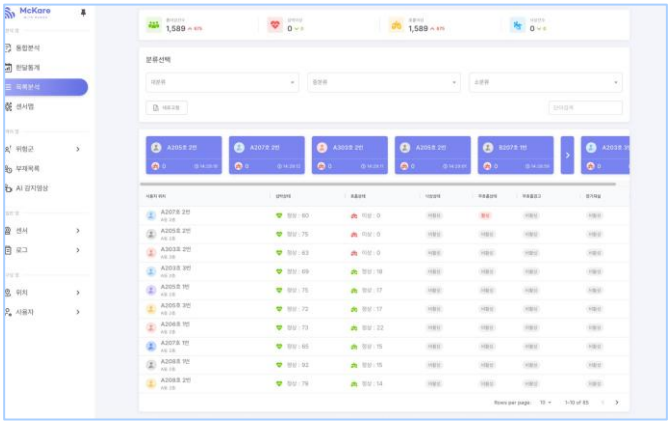
## 1. 맥케어 · 관제 웹 화면 직접 개발한 화면 · 매뉴얼의 데모 데이터



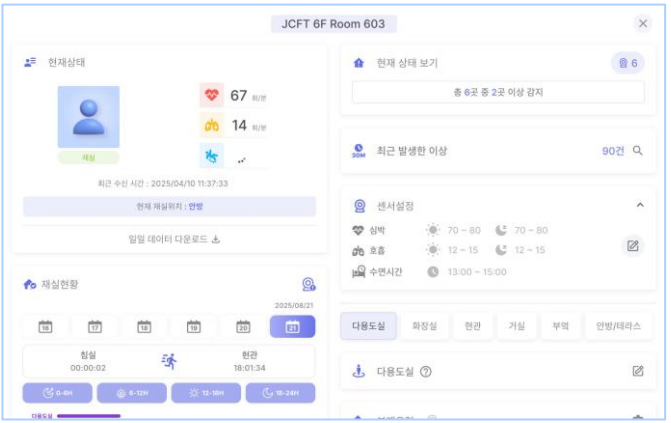
**통합분석** 급증·급감 센서 · 취약시간 순위



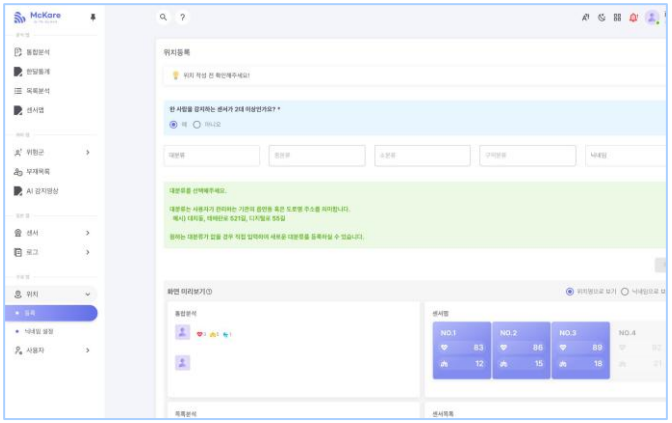
**한달통계** 최근 한달 총계 · 상위 TOP 3



**목록분석** 15초 갱신 · 센서별 상태



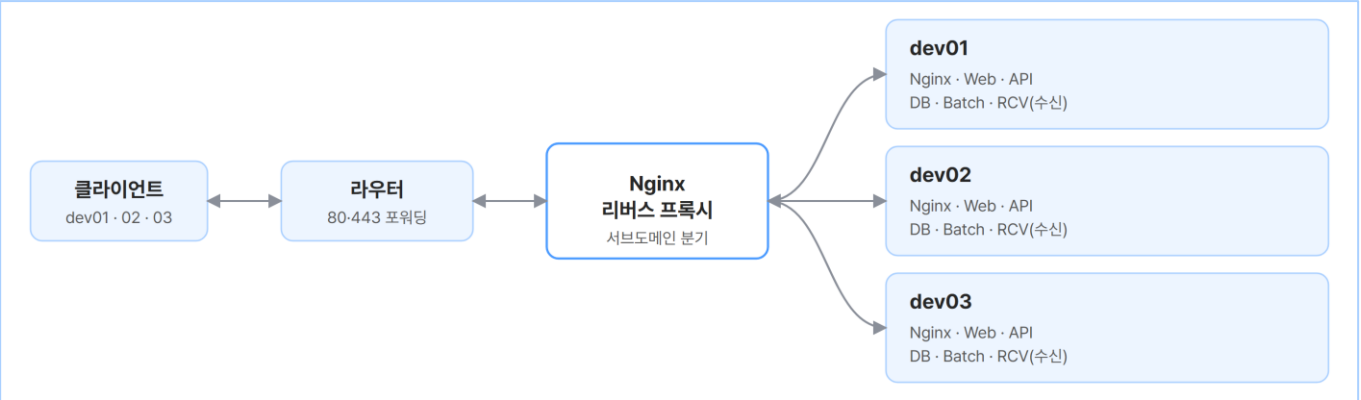

**센서 상세** 재실 타임라인 · 기준값 설정



**위치 등록** 4단계 위치 계층

## 1. 맥케어 · 개발 환경

공인 IP 하나에 80·443 포트만 열고, 리버스 프록시가 서브도메인별로 개발 서버 3개를 나눕니다. 서버마다 Web·API·DB·배치·수신 서버를 컨테이너로 띄워 서로 영향 없이 개발하고 시연할 수 있게 했습니다.




개발 서버 구조. 실제 도메인과 IP는 빼고 새로 그린 그림입니다.

### 트러블슈팅 1 수집 병목을 TCP 단기 연결과 수집 서버 자동 증설로 해결

| | | |
|---|---|---|
| 사례 1 | 상 황 | 센서 2,000대 이상이 5초마다 데이터를 보내자 수집 서버 한 대가 모든 데이터를 받아야 했습니다. |
| | 증 상 | 데이터 누락률 증가, DB 저장 지연, 수집 서버 응답 저하, 로그 누락, 센서 접속 지연 |
| | 원 인 | DB 커넥션을 늘려도 해결되지 않았습니다. 수집 서버 하나가 센서 2,000대 이상의 TCP 연결을 계속 유지하고 센서마다 응답까지 보내느라 데이터를 제때 받지 못한 것이 근본 원인이었습니다. |
| | 해 결 | • 서버가 데이터를 받으면 바로 연결을 끊도록 예외 처리해 TCP 단기 연결로 전환<br>• 센서에 보내던 응답 메시지를 없애고, 연결만 붙잡는 센서는 10초 타임아웃으로 정리<br>• bulk_insert도 비교했지만 이상 감지 지연·유실 위험 때문에 건당 저장 유지, 처리량은 컨테이너 증설로 확보<br>• 5초마다 새로 연결하므로 Nginx가 요청마다 다른 수집 컨테이너로 분산, 부하에 맞춰 컨테이너 자동 추가 |
| | 결 과 | 가상 센서 5만 대(초당 약 1만 건)를 8 vCPU·32GB Spot EC2에서 24·48시간 실행해 누락률 1% 미만을 확인했고, Spot으로 테스트 비용을 82% 줄였습니다. |
| | 지금이라면 | 자동 증설을 Kubernetes 자동 확장으로 옮기고, 수집과 DB 저장 사이에 메시지 큐를 두겠습니다. |

## 트러블슈팅 2 인프라 구성과 약정 방식을 비교해 EC2 비용 30% 절감

| | | |
|---|---|---|
| 사례 2 | 상 황 | 해외 POC가 늘면서 나라마다 서버가 필요해졌고, 어디에 어떤 방식으로 운영할지 정해야 했습니다. |
| | 비 교 | • 온프레미스 vs AWS EC2: 나라마다 서버를 직접 둘 수 없어 EC2로 결정<br>• RDS vs EC2 안의 DB 컨테이너: 인스턴스 하나에 API·수신·DB를 묶어 비용을 낮추고 나라별 일괄 배포<br>• 온디맨드 vs 1년 약정(Saving Plan): 장기 운영 전제로 약정이 더 저렴<br>• RDS 자동 백업 대신 Shell Script로 DB를 tar로 묶어 SFTP로 온프레미스 저장소에 백업 |
| | 설 득 | 구성도와 방식별 비용 비교 자료를 직접 만들어, 영업·대표·팀장이 모인 회의에서 근거를 제시했습니다. |
| | 결 과 | Saving Plan을 도입해 연간 EC2 비용을 약 30% 줄였습니다. |
| | 지금이라면 | 포기한 자동 백업·장애 전환을 비용과 함께 비교해, 서비스가 커지는 시점에 RDS로 옮길 기준을 미리 정해 두겠습니다. |

## 배운 점

| | | |
|---|---|---|
| 배운 점 | 부족했던 점 | 관리자 화면을 "정해진 흐름대로만 쓸 것"이라 보고 가볍게 만들었습니다. 수정·삭제 요청마다 DB에 직접 접속하거나 시스템을 잠시 멈춰야 했고, 전시회 현장에서는 AWS 작업에 시연 시간을 써야 했습니다. |
| | 바꾼 것 | 운영 중 생길 수정·삭제 요청을 미리 예상해 관리자 화면에서 처리하게 만들고, 변경 내용은 모두 로그로 남기며, 중요한 데이터는 특정 권한 계정만 처리하도록 나눴습니다. 운영 안정성은 서버뿐 아니라 관리 화면과 운영 방식까지 포함한다는 걸 배웠습니다. |

## 2. 한화오션 절단공장 야드 관제 시스템 그랜드파워솔루션 · 주임연구원

| | | |
|---|---|---|
| 프로젝트 개요 | 기 간 | 2025.06 – 2026.08 |
| | 역 할 | 개발 담당 · 제조 현장 |
| | 처리 흐름 | 관제 웹·모바일 → API 서버(Java·Spring) → Oracle 프로시저 → 크레인 작업 |
| | 기 술 | Java · Spring · Oracle(SQL Procedure) · Nexacro · Morpheus · JavaScript · GitLab |

| | |
|---|---|
| 프로젝트 설명 | 한화오션 조선소 절단공장 야드에서 크레인 작업과 자재 재고를 관제하는 시스템입니다. 실제 설비를 제어하는 만큼 작업 순서와 안전 요구사항을 정확히 반영해야 했습니다. |

| | |
|---|---|
| 담당 업무 | • Oracle SQL 프로시저로 크레인 작업지시·작업취소·재고설정 제어 로직 개발<br>• Nexacro 기반 야드 관제 웹: 공장 전체 통계와 대시보드, 크레인 작업 현황·재고 화면<br>• Morpheus 기반 모바일: 크레인 차량 작업지시·취소·수정, 사용자 데이터 수정 |

## 3. 한화오션 전처리공장 QR 인식 시스템 그랜드파워솔루션 · 주임연구원

| | | |
|---|---|---|
| 프로젝트 개요 | 기 간 | 2025.06 – 2026.08 |
| | 역 할 | 개발 담당 · 비전 · ERP 연동 |
| | 처리 흐름 | IP 카메라(RTSP) → YOLO 탐지 → 이중 디코딩·감마 보정 → SAP 자재 검증 |
| | 기 술 | Python · OpenCV · YOLO · pyzbar · SAP Interface |

| | |
|---|---|
| 프로젝트 설명 | 전처리공장에서 이송되는 강판의 QR 코드를 카메라로 읽어, ERP(SAP)에 등록된 자재 정보와 일치하는지 검증하는 시스템입니다. |

| | | |
|---|---|---|
| 담당 업무 | 개 발 | • OpenCV로 IP 카메라(RTSP) 실시간 프레임 캡처<br>• 공장 실데이터로 YOLO 기반 QR 영역 탐지 모델 학습<br>• SAP 자재 정보와 QR 데이터 일치 여부 검증 로직 |
| | 인식률 개선 | • pyzbar·OpenCV 이중 디코딩으로 한쪽이 실패해도 다른 쪽으로 읽음<br>• 탐지 영역에 여백을 더해 Quiet Zone 확보<br>• 감마 보정으로 명암 대비를 높이고, Confidence 기준으로 오탐지 제거 |

## 4. K-Cloud 국가사업 GPU 할당 Cloud IDE 그랜드파워솔루션 · 참여

| | | |
|---|---|---|
| 프로젝트 개요 | 기 간 | 2026.07 – 2026.08 |
| | 기 술 | Kubeflow · React · TypeScript · Node.js |
| | 처리 흐름 | 웹 IDE → Kubeflow → GPU 할당 → 모델 학습 검증 |
| | 담당 업무 | • Kubeflow 기반 GPU 자원 할당 웹 IDE 구현 참여<br>• 할당된 환경에서 손글씨 인식 모델을 학습시켜 GPU 할당 동작 검증 |

## 5. 차량부품 비전 불량 검사 한국지능기계 · 학생 체험형 인턴

| | | |
|---|---|---|
| 프로젝트 개요 | 기 간 | 2020.11 – 2021.02 |
| | 기 술 | C# · WinForms · MySQL · PLC 통신 |
| | 처리 흐름 | 카메라 촬영 → ROI·감마 보정 → 불량 판정 → PLC 장비 제어 |
| | 담당 업무 | • 카메라 기반 차량 부품 불량 판정 프로그램 개발<br>• PLC 통신으로 검사 장비의 암전 상태 동작 제어<br>• 영역 미인식 문제를 ROI·감마 보정 알고리즘으로 해결 (이후 QR 인식 개선으로 이어짐) |

## 6. 결제 AIOps 관제 플랫폼 개인 프로젝트 · 실무와 별개

| | | |
|---|---|---|
| 프로젝트 개요 | 기 간 | 2026.09 – 진행 중 |
| | 링 크 | 라이브 데모 aiops.ygrhash.dev · 코드 github.com/ygrhash/Portfolio |
| | 설 명 | 실무에서 깊게 다루지 못한 이벤트 기반 아키텍처를 직접 설계한 프로젝트입니다. 가상 결제 트래픽의 이상 징후를 자동으로 탐지하고 AI(Gemini)가 운영 리포트를 작성합니다. |

| | | |
|---|---|---|
| 설계 · 운영 | 설 계 | • Spring Boot 결제 API가 Kafka로 이벤트 발행, PostgreSQL(영속)과 Redis(실시간 지표)로 역할 분리<br>• Python 이상 탐지·Gemini 리포트, TypeScript 게이트웨이·WebSocket·React 대시보드<br>• AI 리포트 에이전트가 MCP 서버를 도구로 호출해 Kafka Lag·지표·DB 조회 |
| | 운 영 | • k3s와 Terraform으로 AWS(Spot)·Oracle Cloud 배포<br>• Prometheus·Grafana·Tempo 지표·분산 트레이싱, 이상 알림은 n8n을 거쳐 Slack<br>• k6로 단일 노드 안정 한계 약 65~70 TPS 측정(p95 500ms 기준), GitHub Actions CI |